



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI



**POLITECHNIKA
GDAŃSKA**

Laboratorium bonusowe

Michał Czubenko



Fundusze Europejskie
dla Rozwoju Społecznego



Rzeczpospolita
Polska

Dofinansowane przez
Unię Europejską



Cel projektu

Celem projektu jest stworzenie gry inspirowanej klasyczną grą **Tanks 1990 / Battle City** w języku Python z wykorzystaniem biblioteki `pygame` lub `PyQt6`. Projekt musi zostać wykonany zgodnie z zasadami programowania obiektowego. Gra działa wyłącznie w trybie jednoosobowym.

1 Wymagania funkcjonalne

1.1 Mechanika gry

Gra musi zawierać:

- sterowanie czołgiem gracza,
- poruszanie się w czterech kierunkach,
- strzelanie pociskami,
- system kolizji,
- przeciwników AI,
- bazę gracza do obrony,
- system życia,
- system punktów,
- ekran przegranej,
- ekran zwycięstwa,
- kolejne fale przeciwników.

1.2 Architektura OOP

Każdy element gry musi być reprezentowany jako osobny obiekt.

Klasa Environment Odpowiada za:

- inicjalizację `pygame`,
- tworzenie okna gry,
- renderowanie,
- główną pętlę gry,
- aktualizację obiektów,
- obsługę zdarzeń,
- zarządzanie mapą i poziomami.

Klasa `GameObject` Abstrakcyjna klasa bazowa zawierająca:

- pozycję,
- sprite,
- hitbox,
- metodę `update()`,
- metodę `draw()`.

1.3 System czołgów

Należy zaimplementować klasę `Tank`, po której dziedziczą:

- `PlayerTank`,
- `EnemyTank`.

Każdy czołg powinien posiadać:

- punkty życia,
- kierunek ruchu,
- prędkość,
- cooldown strzału,
- poziom ulepszenia,
- liczbę żyć.

1.4 Typy przeciwników

Należy przygotować co najmniej cztery typy przeciwników:

- `BasicTank`,
- `FastTank`,
- `ArmoredTank`,
- `ShooterTank`.

Każdy typ powinien:

- posiadać własne statystyki,
- posiadać własny sprite,
- dziedziczyć po `EnemyTank`.

1.5 AI przeciwników

AI może być zahardkodowane. Wymagane elementy:

- losowy ruch,
- zmiana kierunku przy kolizji,
- okresowe strzelanie,
- podążanie w stronę bazy lub gracza.

1.6 Pociski

Klasa `Bullet` musi:

- poruszać się w kierunku strzału,
- wykrywać kolizje,
- niszczyć przeciwników,
- niszczyć wybrane przeszkody,
- znikać po trafieniu.

1.7 System mapy

Mapa musi być zbudowana z kafelków. Wymagane typy przeszkód:

- `BrickWall`,
- `SteelWall`,
- `Water`,
- `Bush`,
- `Ice`.

Mapy mogą być zahardkodowane.

1.8 Baza gracza

Klasa `Base`:

- może zostać zniszczona,
- kończy grę po trafieniu.

1.9 Power-upy

Należy zaimplementować minimum pięć bonusów:

- Shield,
- SpeedBoost,
- RapidFire,
- ExtraLife,
- Bomb.

Bonusy powinny:

- pojawiać się losowo,
- działać przez ograniczony czas,
- być zbierane przez gracza.

1.10 Obsługa poziomów

Gra powinna:

- obsługiwać minimum trzy poziomy,
- posiadać różne mapy,
- zwiększać poziom trudności,
- zwiększać liczbę przeciwników.

1.11 System kolizji

Kolizje muszą obejmować:

- czołg – ściana,
- czołg – pocisk,
- pocisk – ściana,
- pocisk – pocisk,
- czołg – power-up,
- przeciwnik – baza.

1.12 HUD

Na ekranie należy wyświetlać:

- liczbę żyć,
- wynik,
- liczbę przeciwników,
- aktualny poziom,
- aktywne bonusy.

1.13 Wymagania techniczne

Projekt musi wykorzystywać:

- Python 3.11+,
- bibliotekę pygame lub PyQt6,
- programowanie obiektowe (dziedziczenie, polimorfizm, enkapsulację, abstrakcyjne klasy bazowe).

Przykładowa struktura projektu

```
project/
|
|-- main.py
|-- environment.py
|-- constants.py
|
|-- entities/
|   |-- game_object.py
|   |-- tank.py
|   |-- player_tank.py
|   |-- enemy_tank.py
|   |-- bullet.py
|   |-- base.py
|   |-- powerups.py
|
|-- map/
|   |-- tile.py
|   |-- levels.py
|
|-- ai/
|   |-- enemy_ai.py
|
|-- assets/
|   |-- sprites/
|   |-- sounds/
|
|-- utils/
|   |-- collision.py
```

Ocenianie

Projekt oceniany jest w skali od 0 do 5 punktów.

Kryterium	Punkty
Poprawność działania gry oraz implementacja podstawowych mechanik	3 pkt

Zastosowanie programowania obiektowego i poprawna architektura projektu	1 pkt
Czytelność kodu, organizacja projektu oraz estetyka wykonania	1 pkt
Dodatkowe funkcjonalności i rozszerzenia projektu (np. animacje czy dźwięki)	*1+ pkt

Dodatkowe wymagania

- Kod powinien być czytelny.
- Zabrania się wpisywania wartości liczbowych (progów, granic) – wszystkie parametry należy wyprowadzić z analizy danych. Wyjątkiem jest hardkodowanie plansz.
- Wszystkie linie kodu wygenerowane przez AI powinny być odpowiednio oznaczone. Limit 10% długości kodu.
- Praca musi być samodzielna. Kod będzie weryfikowany pod względem plagiatów jak i użycia AI.